CS 3110 / 5110: Data Privacy — Practice Exam 2 Full name: _____
Fall 2026 This exam is worth a total of **115 points**.

Instructions. Answer each of the following questions. For multiple-choice questions, mark the *one* best answer. For fill-in-the-blank questions, give a single exact value or expression. When a question asks for a privacy cost in (ϵ, δ)-differential privacy, give both ϵ and δ (even if $\delta = 0$). When a question asks for a privacy cost in Rényi differential privacy, give $(\alpha, \bar{\epsilon})$; for zero-concentrated differential privacy, give ρ. For sensitivity and privacy cost answers, feel free to provide the *expression* for calculating the answer—there is no need to actually perform the arithmetic (e.g. if the expression contains variables, square roots, or logarithms). As in the textbook, log denotes the natural logarithm. Unless a question says otherwise, assume that each individual's data is contained in exactly one row of the data (for noisy gradient descent, one training example), and that two datasets are neighbors if one can be obtained from the other by adding or removing one row. In the `retail` dataset, each row is one order (an event), and one customer may have many rows. Under the *event-level* unit of privacy, two datasets are neighbors if one can be obtained from the other by adding or removing one row (one order). Under the *person-level* (*user-level*) unit of privacy, two datasets are neighbors if one can be obtained from the other by adding or removing *all* of the rows of one customer. For questions that contain code, refer to the definitions on the last page of the exam.

1. (5 points) A hospital releases a differentially private histogram of its patients' blood types (8 bins), adding Laplace noise with sensitivity 1 and $\epsilon = 0.5$ to every bin count. A patient worries that releasing 8 noisy counts exposes them to 8 times the privacy risk of releasing a single noisy count. Which response is correct?

   ◯ Yes: the 8 counts are 8 separate releases, so the patient's total privacy cost is $8 \times 0.5 = 4$.

   ◯ No: each count aggregates many patients, so the release has no privacy cost for any one patient.

   ◯ No: the patient's row is in exactly one bin, so only one noisy count depends on their data, for a total cost of 0.5.

   ◯ Yes, unless the hospital gives each bin $\epsilon = 0.5/8$, which brings the total privacy cost back down to 0.5.

2. (5 points) An analyst adds Laplace noise with sensitivity 1 and privacy parameter ϵ to each count in a list of four counts. For which list does parallel composition give a total privacy cost of ϵ?

   ◯ The number of people with age under 30, under 40, under 50, and under 60.

   ◯ The number of people with age 0–29, 30–49, 50–69, and 70 or older.

   ◯ The number of people who are married, who have a bachelor's degree, who work in sales, and who are over 60.

   ◯ The number of people over 60, computed four separate times for four different analysts.

3. (5 points) The homework's `dp_occupation_histogram` applies `laplace_mech(x, 1, epsilon)` to each bin count. Which argument correctly shows that it satisfies ϵ-differential privacy?

   ◯ Each bin is a counting query with sensitivity 1 that gets Laplace noise with scale $1/\epsilon$; by sequential composition the costs of the bins add up to ϵ.

   ◯ Each bin is a counting query with sensitivity 1 that gets Laplace noise with scale $1/\epsilon$; the bins are disjoint, so by parallel composition the total is ϵ.

   ◯ The data is used only once, by the single call to `value_counts`; adding noise to its output is post-processing, so the total is ϵ.

   ◯ Each bin aggregates over a large group of people, so no individual stands out, and the Laplace noise brings the total cost to ϵ.

4. (5 points) An analyst releases `dp_occupation_histogram(0.5)` and *also* releases a contingency table of `Occupation` × `Age` in which every cell count gets `laplace_mech(x, 1, 0.5)`. Both releases are computed from the same `adult` dataset. What is the total privacy cost of the two releases together?

   ϵ: _____

   δ: _____

5. (5 points) A vector-valued query returns the answers to 25 different counting queries. What is its L_2 global sensitivity?

   L_2 global sensitivity: _____

6. (5 points) A vector-valued query returns the answers to 100 counting queries. It is released with gaussian_mech_vec using its L_2 sensitivity, with $\epsilon = 0.5$ and $\delta = 10^{-5}$. What is the standard deviation σ of the Gaussian noise added to each element? (See gaussian_mech on the last page.)

   σ: _____

7. (5 points) An analyst must release the answers to 1,000 counting queries as one vector. The requirements are a total privacy cost of $\epsilon = 1$, a δ of 10^{-5} is acceptable, and the answers should be as accurate as possible. Which design satisfies the requirements and adds the least noise to each answer?

   ◯ laplace_mech_vec with the query's L_2 sensitivity.
   ◯ laplace_mech_vec with the query's L_1 sensitivity.
   ◯ gaussian_mech_vec with the query's L_1 sensitivity.
   ◯ gaussian_mech_vec with the query's L_2 sensitivity.

8. (5 points) In retail_hist_bounded , the contribution bound is raised from 10 to 50, with epsilon unchanged. Which describes the effect?

   ◯ The noise scale grows from $10/\epsilon$ to $50/\epsilon$, but fewer orders are dropped by contribution bounding.
   ◯ The noise scale shrinks from $10/\epsilon$ to $2/\epsilon$, because keeping more orders makes every count larger.
   ◯ The noise scale stays at $10/\epsilon$, but more orders are dropped by contribution bounding.
   ◯ The noise scale grows from $10/\epsilon$ to $50/\epsilon$, and more orders are dropped by contribution bounding.

9. (5 points) A classmate bounds each customer's contribution to 10 rows, but forgets to change the sensitivity:

```
bounded = retail.groupby('CustomerID', group_keys=False).head(10)
country_hist = bounded['Country'].value_counts()
f = lambda x: laplace_mech(x, sensitivity=1, epsilon=0.5)
noisy_hist = country_hist.apply(f)
```

   Assume each customer's orders all come from one country. What is the actual *person-level* privacy cost of releasing noisy_hist ?

   ϵ: _____

   δ: _____

10. (5 points) A store needs the average Quantity per order with a person-level unit of privacy and a total privacy cost of $\epsilon = 1$. The analyst keeps at most 10 orders per customer and clips Quantity to $[0, 100]$. Which design meets the requirement?

    ◯ Noisy sum with sensitivity 1000 and $\epsilon = 1$; noisy count with sensitivity 10 and $\epsilon = 1$; return their quotient.
    ◯ Noisy sum with sensitivity 100 and $\epsilon = 0.5$; noisy count with sensitivity 1 and $\epsilon = 0.5$; return their quotient.
    ◯ Noisy sum with sensitivity 1000 and $\epsilon = 0.5$; noisy count with sensitivity 10 and $\epsilon = 0.5$; return their quotient.

◯ Noisy sum with sensitivity 1000 and $\epsilon = 1$; divide it by the exact number of rows in the bounded data.

11. (5 points) The textbook describes a system with a "person-day" unit of privacy: each day, the data a person submits that day is released with $\epsilon = 0.5$. If a person submits the same data every day for 365 days in a row, what is the effective person-level ϵ for that year, by sequential composition?

    Effective person-level ϵ: _____

12. (5 points) Let queries be a list of 50 counting queries over the adult dataset. What is the total privacy cost of first_big(queries)?

```
def first_big(queries):
    idx = above_threshold(queries, adult, 10000, 0.3)
    return laplace_mech(queries[idx](adult), 1, 0.2)
```

    ϵ: _____

    δ: _____

13. (5 points) An analyst has a total privacy budget of $\epsilon = 1$ and a list queries of 1000 counting queries over *overlapping* groups of people. They want a noisy answer to the first query whose value is above 500. Which design stays within the budget and adds the *least* noise to the released answer?

    ◯ above_threshold with $\epsilon = 0.2$, then laplace_mech(queries[idx](df), 1, 0.8)

    ◯ above_threshold with $\epsilon = 0.5$, then laplace_mech(queries[idx](df), 1, 0.5)

    ◯ laplace_mech(q(df), 1, 0.001) for every query, then return the first noisy answer above 500

    ◯ above_threshold with $\epsilon = 1$, then laplace_mech(queries[idx](df), 1, 0.8)

14. (5 points) Here queries is a list of 200 counting queries over overlapping groups (for example, "how many people are older than a?" for 200 values of a). A student claims that above_10000_naive has a total privacy cost of ϵ, because it returns only one value. Which statement is correct?

```
def above_10000_naive(queries, epsilon):
    for q in queries:
        v = laplace_mech(q(adult), 1, epsilon)
        if v >= 10000:
            return v
    return None
```

    ◯ The claim is correct: only one noisy value is ever returned, so the total privacy cost is ϵ.

    ◯ Each noisy answer computed in the loop is a separate Laplace release, so by sequential composition the cost can reach 200ϵ.

    ◯ The noise is too small: the sensitivity of each query should be 10000, since the threshold is 10000.

    ◯ The claim is correct: the queries are all different, so parallel composition applies to the loop.

15. (5 points) The gradient for one training example is the vector v = np.array([3, 4]). Using the definition of L2_clip on the last page, what does L2_clip(v, 2.5) return?

    Result: _____

16. (5 points) A student's implementation is supposed to have a total privacy cost of $(\alpha, \bar{\epsilon})$-RDP:

```
def ngd_RDP(iterations, alpha, epsilon_bar):
    theta = np.zeros(X_train.shape[1])
    b = 3
    epsilon_bar_count = 0.05 * epsilon_bar
    epsilon_bar_i = 0.95 * epsilon_bar
    noisy_count = gaussian_mech_RDP_vec([len(X_train)], 1, alpha,
                                        epsilon_bar_count)[0]
    for i in range(iterations):
        grad_sum = gradient_sum(theta, X_train, y_train, b)
        noisy_sum = np.array(gaussian_mech_RDP_vec(grad_sum, b, alpha,
                                                   epsilon_bar_i))
        theta = theta - noisy_sum / noisy_count
    return theta
```

Which statement correctly identifies the problem with this implementation?

- ◯ The noisy count should use sensitivity b, because one training example's gradient can be as large as b.
- ◯ Dividing noisy_sum by noisy_count uses the data again, so it adds to the total privacy cost.
- ◯ epsilon_bar_i is not divided by iterations, so the total cost grows with the number of iterations.
- ◯ The noisy gradient sum should use sensitivity 1, because gradient_sum adds one clipped gradient per training example.

17. (5 points) Which statement correctly describes composition in noisy_gradient_descent_zCDP under zero-concentrated differential privacy?

- ◯ The total ρ grows with the square root of the number of iterations, which is why zCDP beats sequential composition.
- ◯ Only the largest ρ counts, as in parallel composition, so each iteration may use the whole budget ρ.
- ◯ The ρs add up, but after converting the total ρ, the resulting ϵ grows more slowly than the number of iterations.
- ◯ The ρs add up, and after converting the total ρ, the resulting ϵ grows in proportion to the number of iterations.

18. (5 points) The functions noisy_gradient_descent, noisy_gradient_descent_RDP (with a well-chosen α), and noisy_gradient_descent_zCDP are run for the same number of iterations. Each is configured so that its total privacy cost, expressed in (ϵ, δ)-differential privacy, is the same (ϵ, δ). Which is likely to produce the *least* accurate model, and why?

- ◯ noisy_gradient_descent, because sequential composition for (ϵ, δ)-differential privacy is loose, so each iteration must add more noise.
- ◯ noisy_gradient_descent_zCDP, because its single parameter ρ cannot be tuned as finely as RDP's α and $\bar{\epsilon}$.
- ◯ noisy_gradient_descent_RDP, because its noise grows with α, while the other two versions do not use α.
- ◯ All three are about equally accurate, because each one adds Gaussian noise to a gradient sum with sensitivity b.

19. (5 points) An analyst runs noisy gradient descent with the same σ in every iteration, and then computes the total privacy cost three ways: with advanced composition for (ϵ, δ)-differential privacy, with RDP composition followed by conversion, and with zCDP composition followed by conversion (using the same δ each time). The three methods report three different values of ϵ for the same trained model. Which explanation is correct?

- ◯ Only the smallest ϵ is correct; the other two methods must contain arithmetic mistakes.
- ◯ Each method gives an upper bound on the same algorithm's privacy cost; RDP and zCDP give tighter bounds.
- ◯ Analyzing the algorithm with zCDP makes the trained model more private than analyzing it with advanced composition.
- ◯ The RDP and zCDP values describe accuracy rather than privacy, so they cannot be compared with the other value.

20. (5 points) You must train a model with 100 iterations of noisy gradient descent, report a total privacy cost of $(1.0, 10^{-5})$-differential privacy, and use zCDP to perform the composition. Which plan meets this requirement while adding the *least* noise?
    - ◯ Call `noisy_gradient_descent_zCDP(100, 1.0)`, since a total of $\rho = 1.0$ corresponds to $\epsilon = 1.0$.
    - ◯ Give each iteration $\rho_i = 0.01$, convert each ρ_i to (ϵ_i, δ_i) separately, and add up the 100 values of ϵ_i.
    - ◯ Find the largest ρ whose converted ϵ is at most 1.0, then call `noisy_gradient_descent_zCDP(100, rho / 100)`.
    - ◯ Find the largest ρ whose converted ϵ is at most 1.0, then call `noisy_gradient_descent_zCDP(100, rho)`.

21. (5 points) In `noisy_gradient_descent_zCDP( iterations , rho)` (5% of `rho` for the count, the rest split evenly across iterations), the total budget `rho` is doubled while `iterations` stays the same. By what factor does the standard deviation σ of the noise added to each gradient sum change?

    Factor: _____

22. (5 points) `noisy_gradient_descent_RDP` and `noisy_gradient_descent_zCDP` are each set up so that, after converting to (ϵ, δ)-differential privacy, their total cost is $(1.0, 10^{-5})$. For the RDP version, the conversion $\epsilon = \bar{\epsilon} + \frac{\log(1/\delta)}{\alpha - 1}$ means that the largest allowed $\bar{\epsilon}$ is about 0.39 with $\alpha = 20$, but only about 0.04 with $\alpha = 13$. If the analyst picks $\alpha = 13$, what is the most likely result?
    - ◯ The RDP version becomes more accurate, because the noise σ grows with α and 13 is smaller than 20.
    - ◯ Both versions add the same amount of noise, because they meet the same (ϵ, δ) target.
    - ◯ The zCDP version also becomes less accurate, because it uses the same α internally.
    - ◯ The RDP version adds much more noise per iteration than the zCDP version, so its model is likely less accurate.

23. (5 points) Using the definition of `noisy_gradient_descent` on the last page and sequential composition, what is the total privacy cost of the following call?

```
noisy_gradient_descent(10, 0.1, 1e-6)
```

    ϵ: _____

    δ: _____

Definitions

Data. The dataframe `adult` has one row per person, with columns including `Age`, `Sex`, `Occupation`, `Education`, `Education-Num`, `Capital Gain`, and `Capital Loss`. The dataframe `retail` has one row per order (an event), with columns `CustomerID`, `Country`, and `Quantity` (number of items). `X_train` and `y_train` contain the training examples (one row per individual) and their labels.

Mechanisms and the sparse vector technique.

```
def laplace_mech(v, sensitivity, epsilon):
    return v + np.random.laplace(loc=0, scale=sensitivity / epsilon)
def laplace_mech_vec(vec, sensitivity, epsilon):
    return [laplace_mech(v, sensitivity, epsilon) for v in vec]
def gaussian_mech(v, sensitivity, epsilon, delta):
    return v + np.random.normal(loc=0,
        scale=sensitivity * np.sqrt(2*np.log(1.25/delta)) / epsilon)
def gaussian_mech_vec(vec, sensitivity, epsilon, delta):
    return [gaussian_mech(v, sensitivity, epsilon, delta) for v in vec]
def gaussian_mech_RDP_vec(vec, sensitivity, alpha, epsilon):
    sigma = np.sqrt((sensitivity**2 * alpha) / (2 * epsilon))
    return [v + np.random.normal(loc=0, scale=sigma) for v in vec]
def gaussian_mech_zCDP_vec(vec, sensitivity, rho):
    sigma = np.sqrt((sensitivity**2) / (2 * rho))
    return [v + np.random.normal(loc=0, scale=sigma) for v in vec]
# preserves epsilon-differential privacy
def above_threshold(queries, df, T, epsilon):
    T_hat = T + np.random.laplace(loc=0, scale = 2/epsilon)
    for idx, q in enumerate(queries):
        nu_i = np.random.laplace(loc=0, scale = 4/epsilon)
        if q(df) + nu_i >= T_hat:
            return idx
    return -1 # the index of the last element
```

Noisy gradient descent. `gradient(theta, xi, yi)` is the gradient of the logistic loss for one example.

```
def L2_clip(v, b):
    norm = np.linalg.norm(v, ord=2)
    if norm > b:
        return b * (v / norm)
    else:
        return v
def gradient_sum(theta, X, y, b):
    gradients = [L2_clip(gradient(theta, x_i, y_i), b) for x_i, y_i in zip(X,y)]
    return np.sum(gradients, axis=0)
def noisy_gradient_descent(iterations, epsilon, delta):
    theta = np.zeros(X_train.shape[1])
    b = 3
    noisy_count = laplace_mech(X_train.shape[0], 1, epsilon)
    for i in range(iterations):
        clipped_gradient_sum = gradient_sum(theta, X_train, y_train, b)
        noisy_gradient_sum = np.array(
            gaussian_mech_vec(clipped_gradient_sum, b, epsilon, delta))
        noisy_avg_gradient = noisy_gradient_sum / noisy_count
        theta = theta - noisy_avg_gradient
    return theta
```

Contribution bounding. The line below keeps at most `bound` rows for each customer (each customer's first rows, in table order):

```
bounded = retail.groupby('CustomerID', group_keys=False).head(bound)
```

Other homework functions. `noisy_gradient_descent_RDP` and `noisy_gradient_descent_zCDP` work like `noisy_gradient_descent`, but use `gaussian_mech_RDP_vec` or `gaussian_mech_zCDP_vec` for the noisy count and for every noisy gradient sum. `retail_hist_bounded` and `retail_avg_quantity` use contribution bounding and have a person-level (user-level) unit of privacy. `dp_occupation_histogram(epsilon)` returns a histogram of `Occupation`, adding `laplace_mech(x, 1, epsilon)` to each count.